Laboratório de Segurança
Análise de Vulnerabilidades em Ambiente Controlado

Vinicius Trindade Dias Abel
Matrícula: 2020007112

Universidade Federal de Minas Gerais (UFMG)
Belo Horizonte – MG – Brasil

viniciustda@ufmg.br

1. Introdução

Este relatório detalha as vulnerabilidades identificadas na aplicação web DVWA (Damn Vulnerable Web Application) em um ambiente de laboratório controlado. O objetivo foi exercitar o ciclo de um teste de intrusão, desde o reconhecimento até a documentação dos achados.

2. Ambiente

- **Distribuição Linux:** Ubuntu 22.04.
- **Navegador:** Firefox.
- **Alvo:** Aplicação web DVWA (Damn Vulnerable Web Application).
- **Endereço:** http://localhost:8080.
- **Preparação do Ambiente:**
  - A aplicação foi executada em um ambiente isolado utilizando um contêiner Docker local.
  - O contêiner foi iniciado com o comando:
    sudo docker run -d --rm --name dvwa -p 8080:80 vulnerables/web-dvwa
  - O nível de segurança da aplicação foi configurado manualmente para "*Low*" através da interface "*DVWA Security*" para a execução dos testes.
- **Serviços Identificados:**
  - A varredura com *nmap* identificou um servidor web *Apache httpd 2.4.25 (Debian)* rodando na porta *8080/tcp*.
  - Os cabeçalhos HTTP, inspecionados com curl, confirmaram a versão do servidor web e o uso de PHP na aplicação.

3. Metodologia

A análise seguiu uma metodologia de teste manual, utilizando ferramentas padrão para reconhecimento e exploração de vulnerabilidades.

- **Varredura de Portas e Serviços:**
  - A ferramenta *nmap* foi utilizada para identificar as portas abertas no host local e as versões dos serviços em execução.
  - Comandos utilizados:
    sudo nmap -sS -p- -T4 127.0.0.1 e *sudo nmap -sV -p 8080 127.0.0.1*
- **Análise de Cabeçalhos HTTP:**
  - A ferramenta *curl* foi empregada para inspecionar os cabeçalhos da resposta HTTP do servidor.
  - Comando utilizado:

`curl -I http://localhost:8080`

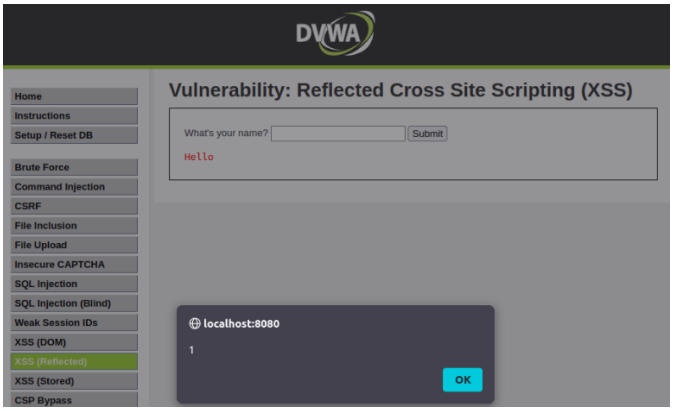
- **Exploração Manual:**
  - O navegador web *Firefox* foi utilizado para interagir diretamente com a aplicação, permitindo a injeção de payloads de teste para comprovar as vulnerabilidades de forma manual.

4. Achados

Foram identificadas três vulnerabilidades críticas na aplicação, exploradas com sucesso no nível de segurança "*Low*".

4.1. Cross-Site Scripting (XSS) Refletido

- **Descrição:** A aplicação reflete a entrada do usuário diretamente na resposta HTML sem a devida codificação, permitindo que um atacante injete código JavaScript. Este código é executado no navegador de um usuário visitante, explorando a confiança do usuário no site vulnerável.
  O teste inicial, com o payload `<script>alert(1)</script>`, foi bem-sucedido no nível de segurança "*Low*". Para aprofundar a análise, foi observado que em níveis de segurança mais altos, filtros básicos de segurança bloqueiam este payload simples. Isso demonstra a importância do contexto da injeção, forçando atacantes a utilizarem payloads variados (ex: `"><script>alert(1)</script>`) para tentar escapar de atributos HTML e contornar os filtros de segurança.
- **Passos para Reproduzir:**
  1. Acessar a página "*XSS (Reflected)*".
  2. Inserir o payload `<script>alert(1)</script>` no campo de entrada.
  3. Clicar em "*Submit*".
  4. Observar que uma caixa de alerta é executada pelo navegador.
- **Impacto:** Permite o roubo de cookies de sessão de usuários, o redirecionamento para sites maliciosos e a manipulação do conteúdo da página no lado do cliente.
- **Gravidade:** Média.
- **Evidência:**



4.2. Injeção de SQL

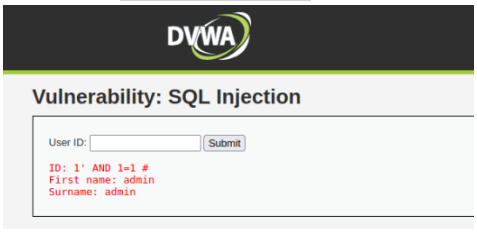
- **Descrição:** A aplicação concatena a entrada do usuário diretamente em uma consulta SQL sem validação, permitindo a manipulação da query. A vulnerabilidade foi confirmada em etapas: primeiro, através de uma técnica *boolean-based*, onde a aplicação respondeu de forma diferente a condições verdadeiras e falsas. Em seguida, a falha foi explorada com um *bypass simples* (`' OR 1=1 #`) para listar todos os registros da tabela atual. Finalmente, um ataque de *UNION SELECT* foi utilizado para extrair metadados sensíveis, como os nomes de todas as tabelas, de todo o banco de dados.

- **Motivo da Exploração:** A query foi executada com sucesso porque a aplicação confia na entrada do usuário e a coloca diretamente em um comando SQL, o que permite manipular o comando original. A quebra da consulta foi realizada usando um apóstrofo para encerrar o comando original e iniciar a injeção. Em seguida, foi adicionado um novo comando, *UNION SELECT*, para pedir ao banco de dados que, além do resultado original, retornasse também a lista com os nomes de todas as tabelas (*table_name*).

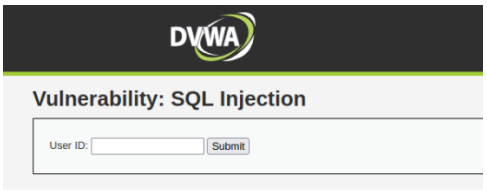
- **Passos para Reproduzir:**
  1. Acessar a página "*SQL Injection*".
  2. Confirmar a falha: Inserir *1' AND 1=1 #* (que retorna o usuário) e depois *1' AND 1=2 #* (que não retorna nada), observando a diferença nas respostas como prova da injeção.
  3. Bypass de listagem: Inserir o payload *' OR 1=1 #* para fazer com que a consulta retorne todos os usuários da tabela.
  4. Extração de metadados: Inserir *' UNION SELECT table_name, NULL from information_schema.tables #* para extrair e exibir os nomes de todas as tabelas do banco de dados.

- **Impacto:** Vazamento completo de dados e da estrutura do banco de dados, o que pode expor informações de usuários, senhas e outros dados sensíveis da aplicação.
- **Gravidade:** Alta.
- **Evidência:**

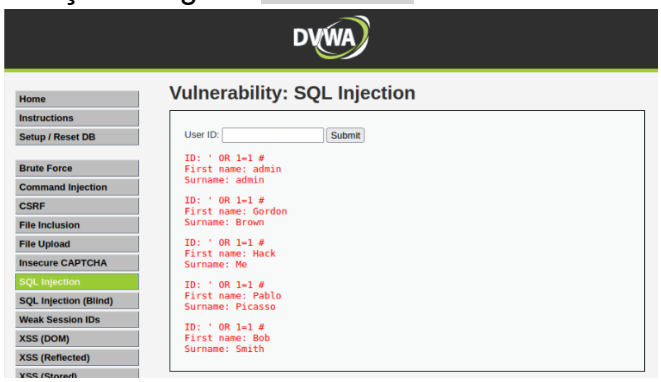
Verdadeiro: *1' AND 1=1 #*



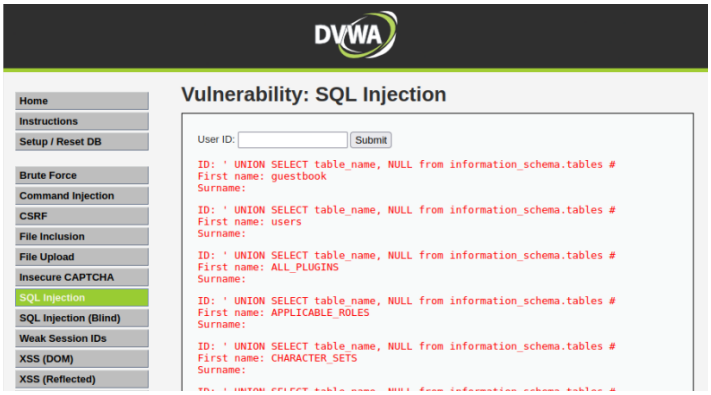
Falso: *1' AND 1=2 #*



Bypass simples de autenticação/listagem: *' OR 1=1 #*



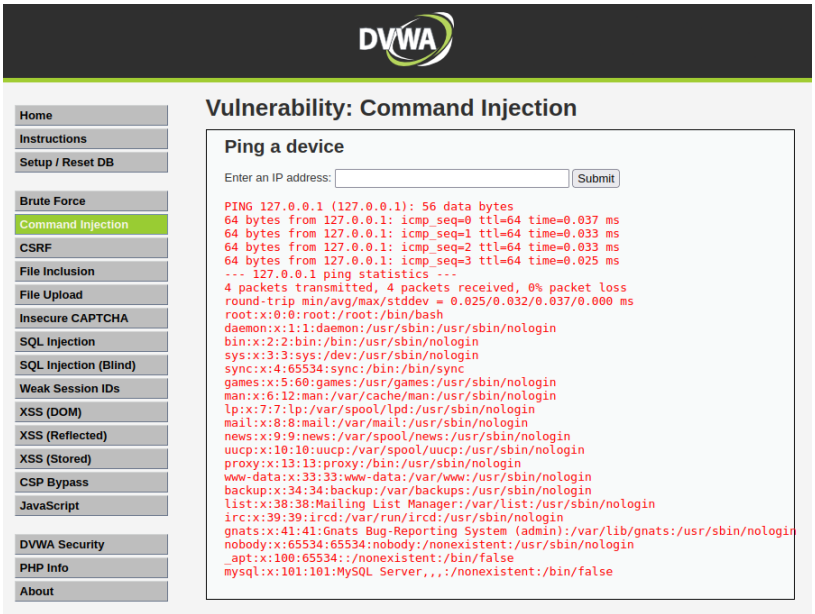
Bypass para listar todas as tabelas no MySQL: *' UNION SELECT table_name, NULL from information_schema.tables #*



4.3. Command Injection

- **Descrição:** A funcionalidade de *ping* permite a execução de comandos arbitrários no sistema operacional do servidor por não validar a entrada do usuário. Um invasor pode encadear comandos usando um separador (*;*), levando ao comprometimento do host.
- **Motivo da Exploração:** A exploração foi possível porque a aplicação executou o comando *ping* com a entrada do usuário sem validar se ela continha apenas um IP. Ao utilizar o ponto e vírgula como separador, foi possível encadear um segundo comando, *cat /etc/passwd*. Este comando instruiu o sistema operacional do servidor a ler o arquivo de sistema */etc/passwd*. Como resultado, o conteúdo deste arquivo foi exibido na página web.
- **Passos para Reproduzir:**
    1. Acessar a página "*Command Injection*".
    2. Inserir o payload *127.0.0.1 ; cat /etc/passwd* no campo de IP.
    3. Clicar em "Submit".
    4. Observar que, além da saída do ping, o conteúdo do arquivo */etc/passwd* do servidor é exibido na tela.
- **Impacto:** Comprometimento total do servidor, permitindo a leitura de arquivos sensíveis, modificação de dados e a potencial instalação de software malicioso.
- **Gravidade:** Alta.

Evidência:



5. Recomendações

Para mitigar as vulnerabilidades identificadas, recomenda-se a adoção de práticas de desenvolvimento seguro:

- **Para XSS Refletido:** Aplicar codificação de saída em todos os dados gerados pelo usuário antes de serem inseridos na página HTML. Isso converte caracteres perigosos (como < e >) em suas entidades HTML equivalentes, neutralizando a capacidade de execução de scripts.
- **Para Injeção SQL:** Utilizar exclusivamente consultas parametrizadas. Esta técnica separa os comandos SQL dos dados, garantindo que a entrada do usuário seja sempre tratada como dados e nunca como código executável.
- **Para Command Injection:** Implementar validação de entrada para permitir apenas os caracteres e o formato esperados (neste caso, um endereço de IP). Evitar o uso de funções que invocam o shell do sistema diretamente com dados do usuário.